

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

К защите допустить:

Заведующий кафедрой ИИТ

_____ Д. В. Шункевич

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
по дисциплине «Математические основы интеллектуальных систем»
на тему:

БГУИР КП4 6-05-0611-03 042 ПЗ

Студент гр. 421701
Проверяющий

М. А. Сухоруков
Н. А. Гуменный

Минск 2026

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет
Специальность

ИТиУ
6-05-0611-03

Кафедра

ИИТ

УТВЕРЖДАЮ

_____ Зав. кафедрой
«____» _____ 2026 г.

ЗАДАНИЕ

по курсовому проекту студента

Крамич Анны Андреевны

(фамилия, имя, отчество)

1. Тема работы: _____

2. Срок сдачи студентом законченной работы: 11.05.2026

3. Исходные данные к проекту: _____

4. Содержание пояснительной записки (перечень подлежащих разработке вопросов):

Введение

1 Анализ подходов к разработке системы автоматизации проверки студенческих работ .

2 Проектирование системы автоматизации проверки студенческих работ .

3 Разработка системы автоматизации проверки студенческих работ .

Заключение

5. Перечень графического материала (с точным указанием обязательных чертежей):
Компьютерная презентация курсового проекта _____

КАЛЕНДАРНЫЙ ПЛАН

п/п	Наименование этапов курсовой работы	Объем этапа, %	Срок выполнения этапов	Примечание
1	Подбор и изучение литературы	10	9.02 – 20.02	
2	Изучение проблемной области, средств проектирования и разработки	10	21.02 – 27.03	
3	Определение требований к базе знаний системы	10	28.02 – 06.03	
4	Проектирование модели базы знаний системы	25	07.03 – 13.04	
5	Реализация базы знаний системы	30	14.04 – 08.05	
6	Оформление пояснительной записки	10	01.05 – 11.05	
7	Оформление графической части проекта	5	06.05 – 11.05	

Дата выдачи задания: 20.02.2026 г. Проверяющий _____ Н. А. Гуменный

Задание принял к исполнению _____ А. А. Крамич

СОДЕРЖАНИЕ

Перечень условных обозначений	4
Введение	5
1 Анализ предметной области и требований к базе знаний	6
1.1 Характеристика предметной области	6
1.2 Типы задач и сценарии использования	6
1.3 Анализ существующих решений и баз знаний	8
1.4 Характеристика аналогичных решений	9
1.5 Анализ требований к базе знаний	10
1.6 Требования к видам знаний и формализму	11
1.7 Вывод по разделу анализа	12
2 Проектирование базы знаний интеллектуальной системы	13
2.1 Общая архитектура интеллектуальной системы и место моду- ля пользовательского интерфейса	13
2.2 Выбор формализма и логической основы	16
2.3 Концептуальная модель данных модуля пользовательского интерфейса	19
2.4 Логическая и физическая структура модуля пользовательско- го интерфейса	21
2.5 Моделирование видов данных и событий в модуле пользова- тельского интерфейса	23
2.6 Обеспечение согласованности, расширяемости и сопровождае- мости	26
2.7 Вывод по разделу проектирования	27
3 Реализация логики анализа и проверки	28
3.1 Выбор средств реализации	28
3.2 Реализация структуры и наполнения логики анализа	28
3.3 Реализация механизмов доступа и использования правил	29
3.4 Демонстрация работы логики анализа	29
3.5 Методика тестирования	30
3.6 Тестовый набор и классификация тестов	30
3.7 Оформление и результаты тестирования	31
3.8 Анализ качества логики анализа	32
3.9 Вывод по разделу реализации	32
Заключение	33
Список использованных источников	33

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

<i>БЗ</i>	– база знаний
<i>ВК</i>	– вики-система
<i>ЭС</i>	– экспертная система
<i>СУЗ</i>	– система управления знаниями
<i>СП</i>	– сообщество практики
<i>AI</i>	– искусственный интеллект
<i>IT</i>	– информационные технологии
<i>KPI</i>	– ключевые показатели эффективности
<i>KM</i>	– управление знаниями
<i>ROI</i>	– возврат инвестиций

ВВЕДЕНИЕ

Современное образование характеризуется активным внедрением цифровых технологий, направленных на повышение эффективности учебного процесса и сокращение времени на выполнение рутинных задач. Одной из таких задач является проверка студенческих работ, которая требует от преподавателя значительных временных затрат. Необходимо анализировать содержание документов, проверять их структуру, соответствие требованиям оформления и фиксировать найденные нарушения. В связи с этим актуальной является разработка интеллектуальных систем, способных автоматизировать отдельные этапы проверки учебных материалов.

Важной частью такой системы является база знаний, в которой хранятся правила, параметры и ограничения, используемые при анализе студенческих работ. Наличие базы знаний позволяет формализовать требования к оформлению документов, обеспечить единообразие проверки и упростить последующее расширение системы. Разработка базы знаний особенно важна в задачах, связанных с классификацией документов, диагностикой нарушений, выбором правил проверки и объяснением полученных результатов.

Целью данного курсового проекта является разработка базы знаний интеллектуальной системы автоматизации проверки студенческих работ.

Для достижения поставленной цели необходимо решить следующие задачи:

- выполнить анализ предметной области и определить основные задачи, решаемые системой;
- выявить требования к базе знаний;
- определить виды знаний и выбрать способ их представления;
- спроектировать структуру базы знаний;
- реализовать базу знаний и продемонстрировать её использование;
- провести тестирование и оценить результаты работы.

Объектом исследования является интеллектуальная система автоматизации проверки студенческих работ. Предметом исследования является база знаний данной системы, включающая правила проверки, способы представления знаний, структуру хранения и особенности использования при анализе документов.

В рамках курсового проекта рассматривается разработка базы знаний, предназначенной для работы с электронными документами в форматах DOCX и PDF. Система должна поддерживать задачи извлечения текста, применения правил проверки, выявления нарушений и формирования результатов анализа. При этом за рамками проекта остаются сложные механизмы интеллектуального вывода, полнофункциональные сетевые сервисы и расширенные средства интеграции с внешними информационными системами.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ТРЕБОВАНИЙ К БАЗЕ ЗНАНИЙ

1.1 Характеристика предметной области

Предметной областью разрабатываемой системы является автоматизация проверки студенческих работ, представленных в электронном виде. К таким работам относятся лабораторные работы, курсовые работы, отчёты и другие учебные документы. Основная цель системы состоит в упрощении процесса проверки и сокращении времени, затрачиваемого преподавателем на анализ работ.

Основными объектами предметной области являются студенческая работа, файл документа, извлечённый текст, правила проверки и результаты анализа. Документ загружается в систему в формате DOCX или PDF, после чего из него извлекается текстовое содержимое. Далее этот текст анализируется по заданным правилам, связанным с оформлением и структурой работы.

К основным задачам предметной области относятся загрузка документов, извлечение текста, применение правил проверки и отображение результатов анализа. Ожидаемым результатом работы системы является выявление нарушений и представление преподавателю удобной информации о состоянии проверяемой работы.

Особенностями предметной области являются необходимость работы с документами различной структуры, требование к точности извлечения текста, а также использование методических указаний и правил оформления в качестве источников знаний ?. Кроме того, система должна обеспечивать приемлемое время обработки и наглядное представление результатов проверки ?.

1.2 Типы задач и сценарии использования

База знаний разрабатываемой системы предназначена для решения задач, связанных с автоматизированной проверкой студенческих работ. В рамках проекта она используется для хранения и применения правил, необходимых при анализе загруженных документов.

К основным типам задач, решаемых с использованием базы знаний, относятся классификация, диагностика, выбор и объяснение результатов проверки ?.

Задача классификации заключается в определении типа проверяемого документа и выборе соответствующего набора правил. Например, в зависимости от вида работы система может использовать разные требования к структуре, объёму и оформлению документа.

Задача диагностики связана с выявлением нарушений в оформлении и структуре студенческой работы. На основе правил, хранящихся в базе знаний, система определяет наличие отклонений от установленных требований, таких как отсутствие обязательных разделов, неправильное оформление заголовков, несоответствие параметров текста или другие ошибки.

Задача выбора состоит в определении правил и параметров, которые должны быть применены к конкретному документу. Выбор зависит от типа работы, заданных требований преподавателя или кафедры, а также особенностей конкретного сценария проверки.

Задача объяснения заключается в формировании понятного для пользователя результата проверки. Система должна не только выявлять нарушения, но и указывать, какое именно правило было нарушено, а также представлять описание ошибки в удобной форме.

С использованием базы знаний могут быть реализованы следующие сценарии:

а) Проверка загруженной работы. Пользователь загружает документ в формате DOCX или PDF, после чего система извлекает текст и передаёт данные модулю анализа. Далее выполняется обращение к базе знаний, из которой выбираются правила, соответствующие данному типу работы. В результате пользователь получает информацию о найденных нарушениях и итогах проверки.

б) Использование наборов требований. Пользователь или администратор задаёт параметры проверки, например, выбирает кафедру, дисциплину или тип работы. После этого система обращается к базе знаний и применяет только тот набор правил, который соответствует заданным условиям. Результатом является более точная и адаптированная проверка документа.

в) Формирование итогового отчёта. После завершения анализа система использует сведения из базы знаний для интерпретации выявленных нарушений и подготовки текстовых сообщений. В результате формируется отчёт, содержащий перечень ошибок и замечаний, представленный в понятной для преподавателя форме.

г) Обновление базы знаний. Администратор или разработчик может добавлять новые правила, изменять существующие требования и уточнять параметры проверки. После обновления базы знаний система получает возможность использовать новые данные без изменения основной логики работы приложения.

База знаний в разрабатываемой системе используется для решения задач классификации, диагностики, выбора и объяснения. Она обеспечивает применение формализованных правил при анализе студенческих работ и позволяет формировать результаты проверки в удобной и понятной для пользователя форме.

1.3 Анализ существующих решений и баз знаний

В области автоматизации проверки студенческих работ существует ряд программных решений, предназначенных для анализа текстовых документов, проверки оригинальности, применения критериев оценивания и формирования отчетов. Такие системы позволяют сократить время проверки, повысить единообразие оценки и упростить работу преподавателя.

С точки зрения решаемых задач можно выделить несколько типовых подходов к автоматизации проверки студенческих работ, различающихся назначением системы, способом хранения знаний и формой представления результатов.

Таблица 1.1 – Анализ структур баз знаний существующих решений

Система	Назначение	Преимущества	Ограничения	Применение
Turnitin	Проверка текстов на заимствования и формирование отчетов о совпадениях	Высокая эффективность поиска совпадений, удобные отчеты, широкое применение в образовании	Основной акцент сделан на плагиат, а не на оформление и структуру документа	Высокая для анализа оригинальности
Gradescope	Автоматизация проверки заданий и применение критериев оценивания	Удобство использования рубрик, сокращение времени проверки, поддержка разных типов заданий	Ориентирован в первую очередь на оценивание, а не на проверку оформления документа	Высокая для формализованной оценки
PlagScan	Проверка документов на текстовые совпадения и работа с отчетами	Простота использования, интеграция с образовательными платформами	Ограниченная функциональность в части анализа структуры и форматирования	Средняя для проверки заимствований
Локальные системы проверки документов	Контроль структуры, оформления и параметров текстовых файлов	Возможность адаптации под требования конкретной организации	Ограниченность набора правил и отсутствие универсального подхода	Высокая для прикладных локальных решений

Из приведённого анализа можно сделать вывод, что существующие решения, как правило, ориентированы либо на проверку заимствований, либо на поддержку процесса оценивания. При этом задача комплексной автоматической проверки студенческих работ, включающая анализ структуры документа, параметров оформления и формирование понятного отчёта, реализуется не во всех системах в полном объёме.

1.4 Характеристика аналогичных решений

Существующие системы автоматизированной проверки студенческих работ различаются не только по функциональному назначению, но и по способу организации базы знаний, лежащей в их основе.

Система Turnitin предназначена для выявления текстовых заимствований в документах. Принцип её работы основан на сравнении содержания проверяемого документа с внутренними и внешними источниками. База знаний данной системы имеет преимущественно документный характер и представляет собой крупное хранилище текстовых документов, их представлений, а также метаданных и результатов анализа совпадений. Такая структура эффективна для поиска заимствований, однако менее удобна для хранения формализованных правил оформления студенческих работ ?.

Система Gradescope применяется для автоматизации оценивания студенческих работ. В её основе лежит более структурированная база знаний, включающая критерии оценивания, рубрики, параметры заданий и шаблоны проверки. Такая форма представления знаний обеспечивает удобство сопровождения и изменения критериев, а также единообразие оценивания. Однако основное назначение системы связано с процессом оценивания, а не с детальной проверкой структуры и оформления документа ?.

Система PlagScan, аналогично Turnitin, ориентирована на проверку документов на наличие текстовых совпадений. Её база знаний также строится вокруг текстовых источников, результатов сравнения и отчётов о совпадениях. Данный подход обеспечивает высокую эффективность при анализе оригинальности текста, но характеризуется слабой ориентированностью на проверку структуры и форматирования документов ?.

Кроме того, существуют локальные системы проверки документов, разрабатываемые для нужд конкретной организации или учебной задачи. Для таких решений характерно использование таблиц параметров, наборов правил и шаблонов сообщений о нарушениях. В отличие от рассмотренных систем, их базы знаний ориентированы на хранение формализованных требований к оформлению, ограничений по структуре документа и правил проверки. Это делает их наиболее близкими к задачам разрабатываемой системы ?.

Рассмотрим сильные и слабые стороны существующих подходов. Документно-ориентированные базы знаний обладают высокой полнотой в части хранения текстовых материалов и хорошо подходят для поиска совпадений. Их сильной стороной является масштабируемость и удобство анализа текстов, а слабой — недостаточная формализация правил оформления.

Структурированные базы знаний, основанные на рубриках, правилах и параметрах проверки, удобны для сопровождения, расширения и применения в автоматическом анализе. Их преимуществом является воз-

возможность явного описания критериев, однако они требуют предварительной формализации предметной области ?.

Комбинированные подходы, сочетающие хранение текстовых данных, параметров проверки и формализованных правил, являются наиболее подходящими для систем автоматизации проверки студенческих работ. Они позволяют учитывать как содержимое документа, так и его структурные и оформительские характеристики ?.

Проведённый анализ показывает, что для разрабатываемой системы наиболее целесообразно использовать комбинированный подход к построению базы знаний. В такой базе должны храниться правила оформления студенческих работ, параметры проверки, ограничения на структуру документа, сведения о типовых нарушениях и шаблоны формирования отчётов. Это позволит реализовать автоматическую проверку документов не только с точки зрения текстового содержания, но и с точки зрения соответствия установленным требованиям оформления.

1.5 Анализ требований к базе знаний

База знаний системы должна обеспечивать хранение, структурирование и использование правил, необходимых для автоматизированной проверки студенческих работ на соответствие требованиям оформления и критериям анализа текстовых заимствований. Она должна поддерживать применение формализованных правил к загружаемым документам и обеспечивать формирование результатов проверки.

База знаний должна обеспечивать:

- хранение правил оформления студенческих работ, включая требования к структуре документа, форматированию текста, оформлению заголовков, списков, таблиц, рисунков и списка литературы ?;
- хранение критериев проверки текстовых заимствований и параметров оценки оригинальности работы;
- представление правил в формализованном виде, пригодном для автоматической обработки системой ?;
- использование различных наборов правил в зависимости от типа работы (лабораторная работа, курсовая работа, отчёт по практике и др.);
- поддержку настройки правил проверки под требования конкретной кафедры, дисциплины или преподавателя;
- предоставление описания нарушений и соответствующих рекомендаций по их исправлению;
- формирование данных для построения итогового отчёта о результатах проверки;
- обновление и редактирование набора правил без необходимости изменения программного кода системы;

- хранение версий правил и возможность использования актуального набора требований при выполнении проверки;
- поддержку взаимодействия с модулями анализа документа, извлечения текста и визуализации результатов проверки.

К базе знаний предъявляются требования, обеспечивающие корректность, согласованность и эффективность функционирования системы автоматизированной проверки студенческих работ.

База знаний должна удовлетворять следующим требованиям:

- полнота представления знаний: база знаний должна содержать все основные правила и критерии, необходимые для проверки оформления и анализа содержания студенческих работ;
- непротиворечивость: содержащиеся в базе знаний правила не должны конфликтовать друг с другом в пределах одной предметной области или одного типа документа;
- актуальность: должна быть обеспечена возможность своевременного обновления требований в соответствии с действующими методическими указаниями;
- расширяемость: структура базы знаний должна позволять добавление новых правил, категорий требований и типов документов без изменения общей архитектуры системы;
- сопровождаемость: база знаний должна быть удобна для редактирования, проверки и сопровождения со стороны администратора или эксперта предметной области;
- производительность: обращение к базе знаний и применение правил в ходе проверки должны выполняться за приемлемое время;
- настраиваемость: база знаний должна поддерживать адаптацию под различные учебные подразделения и варианты методических требований;
- надёжность: база знаний должна обеспечивать сохранность правил и устойчивость к ошибкам при обновлении и использовании данных;
- понятность интерпретации: правила и результаты их применения должны быть представлены в форме, удобной для объяснения пользователю.

1.6 Требования к видам знаний и формализму

База знаний должна содержать различные виды знаний, необходимые для выполнения автоматической проверки студенческих работ и интерпретации её результатов.

В системе должны использоваться следующие виды знаний:

- декларативные знания: сведения о требованиях к оформлению документов, допустимых параметрах форматирования, обязательных структурных элементах работы и критериях оценки оригинальности текста;
- структурные знания: описание состава документа, взаимосвязей

между его разделами и классификации типов студенческих работ;

- процедурные знания: правила применения критериев проверки, последовательность анализа документа и логика формирования замечаний;
- знания об ограничениях: допустимые диапазоны параметров оформления, обязательные и запрещённые элементы, условия применимости конкретных правил;
- интерпретационные знания: шаблоны пояснений, используемые при формировании сообщений об обнаруженных нарушениях.

Для представления и обработки знаний целесообразно использовать следующие формализмы:

- структурированные записи о правилах проверки;
- таблицы параметров и ограничений для различных типов документов;
- продукционные правила вида «условие – действие»;
- метаданные, определяющие область применимости конкретного правила;
- шаблоны текстовых сообщений для формирования итогового отчёта о результатах проверки.

Используемые формализмы должны обеспечивать возможность формального описания требований к студенческим работам, их автоматического применения при анализе документов, а также удобство расширения и сопровождения базы знаний в процессе эксплуатации системы.

1.7 Вывод по разделу анализа

По результатам анализа можно сделать следующие выводы:

- предметная область связана с автоматизацией проверки студенческих работ в электронном виде;
- база знаний должна использоваться для хранения правил оформления, параметров проверки и условий формирования результатов анализа;
- основными задачами системы являются классификация документов, выявление нарушений и представление результатов проверки;
- анализ аналогов показал необходимость создания решения, поддерживающего не только проверку заимствований, но и контроль структуры и оформления работ;
- при проектировании базы знаний необходимо учитывать требования полноты, актуальности, непротиворечивости, расширяемости и удобства сопровождения;
- в качестве основы для проектирования будут использоваться структурированные правила и параметры, применяемые при автоматическом анализе загружаемых документов.

2 ПРОЕКТИРОВАНИЕ БАЗЫ ЗНАНИЙ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ

2.1 Общая архитектура интеллектуальной системы и место модуля пользовательского интерфейса

Интеллектуальная система автоматизации проверки студенческих работ строится по модульному принципу и включает несколько взаимосвязанных подсистем, обеспечивающих полный цикл обработки документа: от его загрузки до формирования итогового отчёта. Общая архитектура системы ориентирована на разделение пользовательского взаимодействия, логики анализа, работы с базой знаний и хранения результатов.

В состав основных подсистем системы входят:

- модуль пользовательского интерфейса, обеспечивающий взаимодействие студента или преподавателя с системой;
- подсистема загрузки документа, выполняющая приём файла и его сохранение в хранилище документов;
- подсистема анализа документа, координирующая выполнение интеллектуальной обработки загруженного файла;
- подсистема определения типа документа, устанавливающая, к какому классу учебных документов относится загруженный файл;
- подсистема определения семестра, определяющая, под какой семестр подходит анализируемый документ;
- подсистема проверки требований, сопоставляющая параметры документа с правилами, хранящимися в базе знаний;
- подсистема формирования отчёта, подготавливающая итоговые результаты проверки в форме, удобной для пользователя;
- база знаний правил проверки, содержащая сведения о типах документов, семестрах, требованиях, нарушениях, правилах проверки и шаблонах интерпретации результатов;
- хранилище документов, предназначенное для сохранения исходных файлов;
- хранилище результатов, предназначенное для сохранения результатов анализа и сформированных отчётов.

Работа системы начинается с действий пользователя в интерфейсе. Через модуль пользовательского интерфейса выполняются выбор и загрузка документа, запуск анализа и получение результатов проверки. После загрузки файл передаётся в подсистему загрузки, сохраняется в хранилище документов и становится доступным для дальнейшей обработки. Затем подсистема анализа инициирует определение типа документа, определение семестра и проверку соответствия установленным требованиям. Для выполнения этих задач интеллектуальные модули обращаются к базе зна-

ний, где хранятся сведения о допустимых структурах документов, наборах правил для различных типов работ, требованиях к семестрам и шаблонах нарушений. По завершении анализа результаты сохраняются в хранилище результатов и используются подсистемой формирования отчёта.

Архитектурная схема верхнего уровня приведена на рисунке 2.1. На данной схеме показаны основные слои системы, интеллектуальный контур, хранилища и место пользовательского интерфейса в общей структуре приложения.



Рисунок 2.1 – Архитектурная схема верхнего уровня

С точки зрения пользователя работа системы определяется набором основных сценариев использования. Студент может загрузить документ, запустить его анализ и получить итоговый отчёт. Преподаватель также может запускать анализ документа и просматривать результаты проверки. При этом сценарий анализа включает вызов внутренних функций определения типа документа, определения семестра и проверки соответствия требованиям. Соответствующая диаграмма вариантов использования приведена на рисунке 2.2.



Рисунок 2.2 – Диаграмма вариантов использования системы анализа документов

Временной порядок взаимодействия пользователя с системой через модуль пользовательского интерфейса показан на диаграмме последовательности, представленной на рисунке 2.3. На диаграмме отражён сценарий выбора и загрузки документа, передачи файла в подсистему загрузки, запуска анализа, получения результатов обработки и отображения итогового отчёта пользователю. Таким образом, диаграмма последовательности показывает, каким образом интерфейс связывает действия пользователя с внутренними модулями системы и обеспечивает доступ к основным функциям анализа документа.

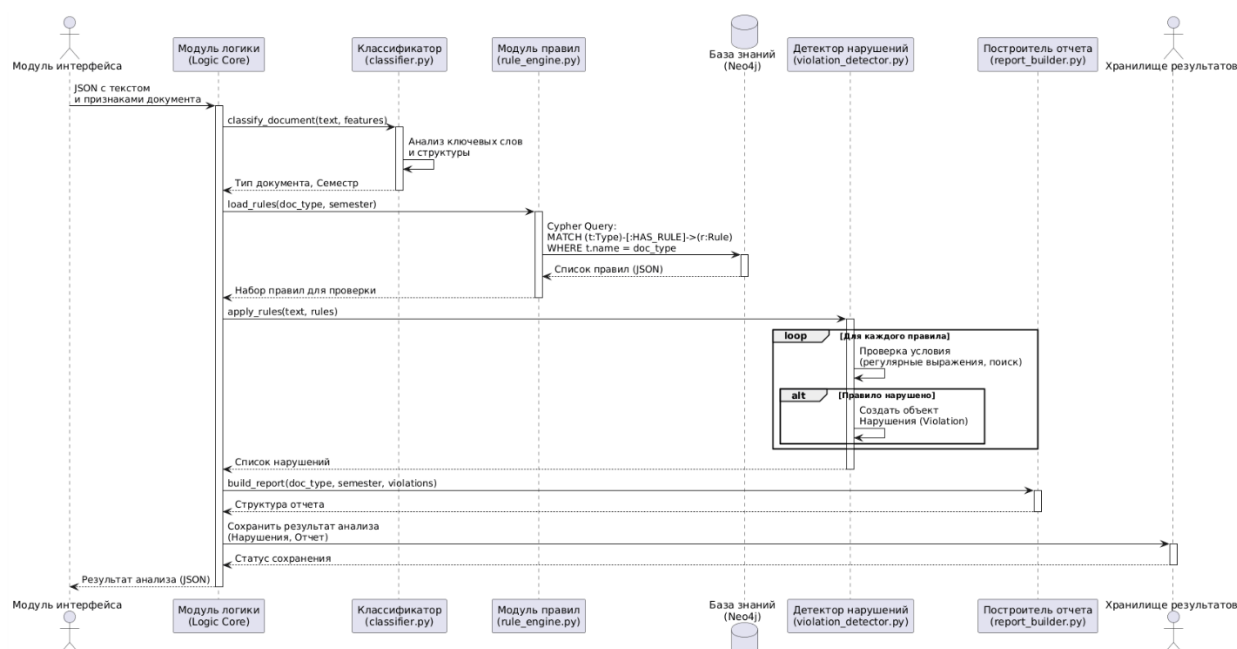


Рисунок 2.3 – Диаграмма последовательности взаимодействия пользователя с системой через интерфейс

Более детально внутренняя организация пользовательского интерфейса представлена на диаграмме компонентов, приведённой на рисунке 2.4. Внутри модуля пользовательского интерфейса выделяются главное окно

интерфейса, компонент выбора и загрузки файла, компонент управления запуском анализа, компонент отображения состояния документа, компонент отображения результатов проверки, компонент просмотра итогового отчёта, а также компонент обработки ошибок и уведомлений.

Через указанные компоненты интерфейс взаимодействует с внешними подсистемами загрузки документа, анализа документа и формирования отчёта. Подсистема загрузки обеспечивает передачу и сохранение документа в хранилище документов. Подсистема анализа выполняет интеллектуальную обработку и передаёт результаты обратно в интерфейс. Подсистема формирования отчёта использует результаты анализа, сохранённые в хранилище результатов, и возвращает пользователю готовый отчёт. Такое разделение позволяет отделить пользовательское взаимодействие от внутренней логики работы системы и сделать интерфейс самостоятельным модулем в общей архитектуре.

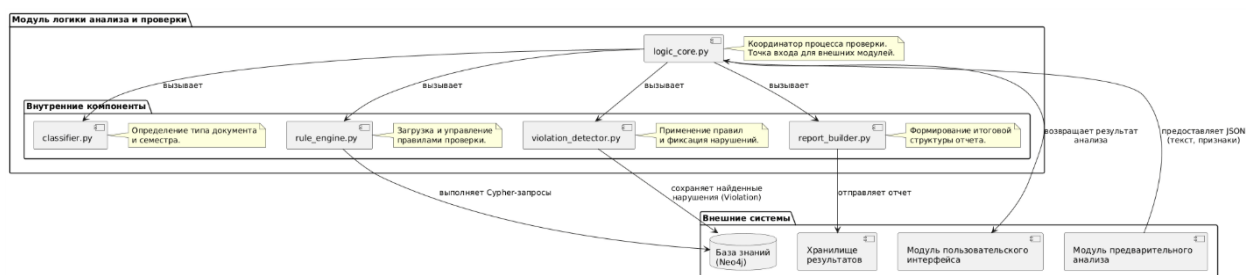


Рисунок 2.4 – Диаграмма компонентов модуля пользовательского интерфейса

Таким образом, общая архитектура интеллектуальной системы автоматизации проверки студенческих работ основана на разделении функций между пользовательским интерфейсом, подсистемами анализа и базой знаний. Модуль пользовательского интерфейса обеспечивает удобное взаимодействие пользователя с системой и предоставляет доступ к функциям загрузки документа, запуска анализа, отображения состояния обработки, просмотра результатов проверки и получения итогового отчёта.

2.2 Выбор формализма и логической основы

Описание выбранного формализма

При проектировании модуля пользовательского интерфейса используется комбинированный подход к представлению данных и взаимодействиям, включающий:

- графовую модель знаний как основное внутреннее представление сущностей и связей интеллектуальной системы;
- структурированные форматы JSON для обмена данными между модулем пользовательского интерфейса и внутренними подсистемами системы;

- событийную модель взаимодействия, определяющую последовательность пользовательских действий и реакций интерфейса;
- исходные документы в форматах PDF и DOCX как входные объекты, загружаемые пользователем через интерфейс.

Выбор такого подхода обусловлен особенностями разрабатываемой системы. Пользовательский интерфейс не является самостоятельным хранилищем знаний, однако он должен обеспечивать корректную работу с результатами, формируемыми на основе базы знаний. Внутренние подсистемы опираются на графовую модель знаний, в которой представлены основные сущности предметной области: документ, тип документа, семестр, требование, проверка, нарушение и отчёт. Для интерфейса важно, чтобы результаты работы этих подсистем были представлены в форме, пригодной для передачи, обработки и отображения.

В качестве транспортного формата взаимодействия между интерфейсом и внутренними модулями системы используется JSON. Такой формат удобен тем, что позволяет передавать структурированные данные о состоянии документа, результатах анализа, найденных нарушениях и сформированном отчёте в компактной и однозначной форме. Использование JSON обеспечивает слабую связанность интерфейсного модуля с остальными частями системы и упрощает последующую интеграцию.

Событийная модель взаимодействия применяется для описания логики работы интерфейса. В рамках данной модели действия пользователя, такие как выбор файла, запуск анализа, запрос отчёта или обработка ошибок, рассматриваются как события, на которые интерфейс должен корректно реагировать. Такой подход хорошо соответствует задачам проектирования пользовательского взаимодействия, поскольку позволяет формально описать последовательность переходов между состояниями интерфейса и взаимосвязь между пользовательскими действиями и ответами системы.

Таким образом, выбранный подход сочетает внутреннее графовое представление знаний всей системы, структурированный обмен данными в формате JSON и событийную логику работы пользовательского интерфейса. Это обеспечивает удобство интеграции интерфейсного модуля с интеллектуальной системой, наглядность отображения результатов и возможность дальнейшего расширения функциональности.

Синтаксис и семантика

В рамках выбранного подхода используются следующие основные синтаксические и семантические элементы:

- *сущность* — объект предметной области, информация о котором должна быть доступна через интерфейс, например документ, тип документа, семестр, нарушение или отчёт;
- *атрибут* — отдельное свойство сущности, например имя файла, формат документа, статус обработки, тип документа, семестр или текст

сообщения об ошибке;

- *событие* — действие пользователя или системы, инициирующее изменение состояния интерфейса, например загрузка файла, запуск анализа, получение результатов или возникновение ошибки;

- *состояние* — текущее состояние документа или интерфейса, например документ не загружен, документ загружен, выполняется анализ, результаты получены, отчёт сформирован;

- *сообщение* — структурированные данные, передаваемые между интерфейсом и внутренними подсистемами, например запрос на запуск анализа или ответ с результатами проверки;

- *ошибка* — специальный тип сообщения, информирующий интерфейс о невозможности корректного завершения операции.

Семантика данного подхода заключается в следующем. Пользователь выполняет действия через элементы интерфейса, которые порождают события. Эти события приводят к формированию запросов к внутренним подсистемам системы. В ответ интерфейс получает структурированные сообщения, содержащие результаты анализа, сведения о состоянии документа или информацию об ошибках. На основании полученных данных интерфейс изменяет своё внутреннее состояние и обновляет отображаемую пользователю информацию.

Для обеспечения корректности взаимодействия вводятся следующие правила:

- каждый документ, передаваемый на анализ, должен иметь уникальный идентификатор или уникальный набор атрибутов, позволяющий однозначно его различать;

- каждое сообщение, передаваемое между интерфейсом и внутренними подсистемами, должно иметь фиксированную структуру;

- статусы обработки документа должны принадлежать заранее определённому множеству допустимых состояний;

- результаты анализа должны включать обязательные поля, необходимые для отображения в интерфейсе;

- сообщения об ошибках должны передаваться в форме, пригодной для однозначной интерпретации и показа пользователю;

- идентификаторы сущностей, используемые в интерфейсе, должны совпадать с идентификаторами соответствующих объектов во внутренних подсистемах системы.

Взаимодействие между интерфейсом и внутренними модулями строится на согласовании структуры данных. Например, информация о документе, его типе, семестре, нарушениях и отчёте должна передаваться в таком виде, чтобы интерфейс мог напрямую использовать её для отображения без дополнительного логического вывода. Благодаря этому модуль пользовательского интерфейса остаётся отделённым от внутренней реализации

базы знаний, но при этом может корректно работать с результатами её применения.

Таким образом, логическая основа модуля пользовательского интерфейса определяется не только общим формализмом базы знаний интеллектуальной системы, но и правилами представления, передачи и интерпретации данных, необходимых для организации пользовательского взаимодействия.

2.3 Концептуальная модель данных модуля пользовательского интерфейса

Концептуальная модель данных модуля пользовательского интерфейса отражает основные сущности, с которыми пользователь взаимодействует в процессе работы с системой, а также связи между ними. В отличие от общей модели предметной области, данная ER-диаграмма ориентирована на те объекты, которые участвуют в пользовательском сценарии загрузки документа, запуска анализа, отображения результатов проверки и получения итогового отчёта.

В состав модели входят сущности «Пользователь», «Документ», «Проверка», «РезультатПроверки», «Нарушение», «Отчёт», «ТипДокумента» и «Семестр». Такое представление позволяет описать данные, необходимые для работы интерфейса, без избыточного включения внутренних сущностей, относящихся исключительно к логике базы знаний или реализации правил проверки.

Центральной сущностью модели является «Документ», поскольку именно он выступает основным объектом пользовательского взаимодействия. Пользователь загружает документ в систему, после чего документ проходит процедуру проверки. Сущность «Проверка» фиксирует факт выполнения анализа, а сущность «РезультатПроверки» содержит сведения, которые в дальнейшем отображаются в пользовательском интерфейсе, включая тип документа, определённый семестр и комментарии по результатам обработки.

С сущностью «РезультатПроверки» связаны сущности «Нарушение» и «Отчёт». Нарушения отражают выявленные несоответствия и рекомендации по их исправлению, а отчёт представляет итоговую форму представления результатов анализа. Дополнительно результат проверки связан с сущностями «ТипДокумента» и «Семестр», поскольку данные об этих объектах выводятся пользователю в интерфейсе после завершения анализа.

На рисунке 2.5 представлена ER-диаграмма данных модуля пользовательского интерфейса.

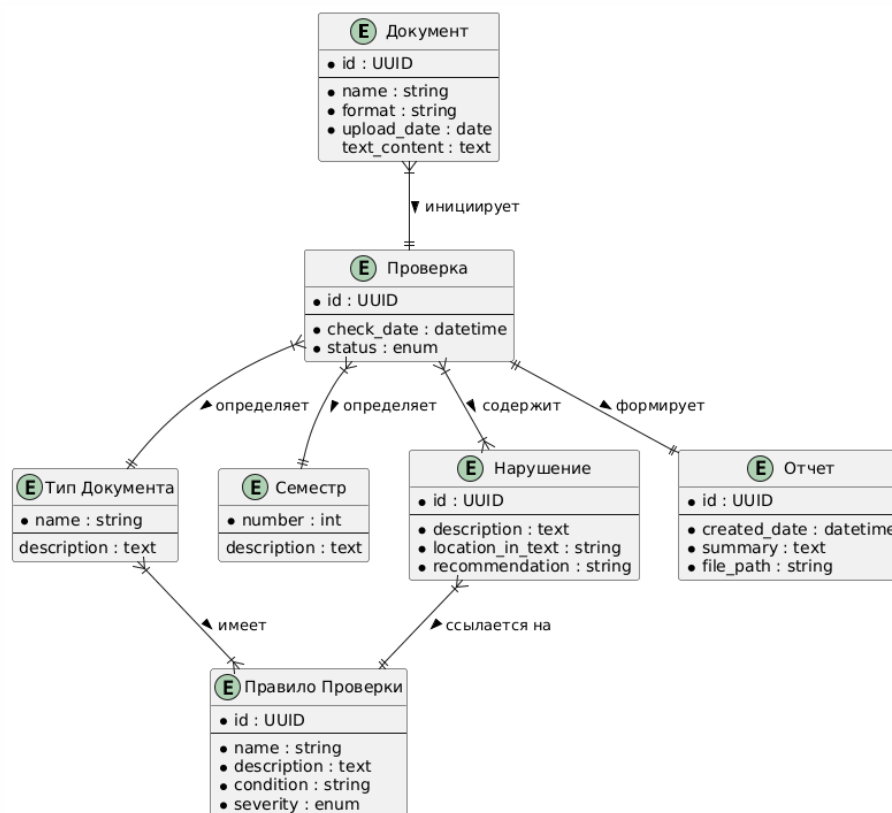


Рисунок 2.5 – ER-диаграмма данных модуля пользовательского интерфейса

В построенной модели используются следующие основные связи:

- пользователь загружает документ;
- документ проходит проверку;
- проверка формирует результат проверки;
- результат проверки содержит нарушения;
- результат проверки оформляется в отчёт;
- результат проверки определяет тип документа;
- результат проверки определяет семестр.

Представленная модель поддерживает задачи, решаемые модулем пользовательского интерфейса, следующим образом. Сущность «Документ» используется для представления загруженного файла и его текущего статуса. Сущность «Проверка» фиксирует выполнение анализа. Сущность «РезультатПроверки» объединяет сведения, подлежащие отображению пользователю. Сущности «Нарушение» и «Отчёт» обеспечивают вывод детализированных результатов проверки и итогового документа. Наличие сущностей «ТипДокумента» и «Семестр» позволяет представить пользователю результаты классификации документа в понятной форме.

Таким образом, концептуальная модель данных модуля пользовательского интерфейса описывает структуру объектов, отображаемых пользователю в процессе взаимодействия с системой, и служит основой для проектирования экранных форм, логики отображения результатов и орга-

низации обмена данными с внутренними подсистемами.

2.4 Логическая и физическая структура модуля пользовательского интерфейса

Логическая структура

Логическая структура модуля пользовательского интерфейса строится как совокупность взаимосвязанных компонентов, каждый из которых отвечает за определённый этап пользовательского взаимодействия с системой. Такое разделение позволяет отделить элементы управления, отображения данных и обработки событий друг от друга, что упрощает сопровождение и дальнейшее развитие интерфейса.

В логической структуре модуля пользовательского интерфейса выделяются следующие основные компоненты:

- *главное окно интерфейса*, объединяющее основные элементы управления и визуального представления данных;
- *компонент выбора и загрузки файла*, обеспечивающий выбор документа и его передачу в подсистему загрузки;
- *компонент управления запуском анализа*, отвечающий за передачу команды на запуск обработки документа;
- *компонент отображения состояния документа*, показывающий текущий этап обработки и статус выполнения операций;
- *компонент отображения результатов проверки*, предназначенный для вывода типа документа, семестра, нарушений и иных результатов анализа;
- *компонент просмотра итогового отчёта*, обеспечивающий доступ пользователя к сформированному отчёту;
- *компонент обработки ошибок и уведомлений*, предназначенный для отображения сообщений о нестандартных ситуациях и ходе выполнения операций.

Каждый из указанных компонентов работает с определённым видом данных. Компонент загрузки файла использует сведения о документе и его формате. Компонент отображения состояния работает со статусами обработки. Компонент результатов проверки использует данные сущности «РезультатПроверки» и связанных с ней объектов. Компонент просмотра отчёта работает с итоговыми сведениями об отчёте. Компонент уведомлений использует сообщения об ошибках и служебные уведомления системы.

Связь между компонентами модуля интерфейса осуществляется через события пользовательского взаимодействия и структурированные данные, поступающие от внутренних подсистем системы. Главное окно интерфейса координирует работу остальных компонентов и обеспечивает их согласованное функционирование.

Границы ответственности компонентов определяются по их роли в пользовательском сценарии. Компонент загрузки отвечает только за выбор и передачу файла. Компонент запуска анализа отвечает за инициацию проверки. Компонент отображения состояния отвечает за визуализацию текущего этапа обработки. Компонент результатов отображает итоговые данные анализа. Компонент отчёта отвечает за просмотр итогового документа. Такое разделение делает структуру интерфейсного модуля прозрачной и пригодной для поэтапного развития.

Физическая структура и форматы хранения

На физическом уровне модуль пользовательского интерфейса реализуется как отдельная часть программного проекта, включающая файлы экранных форм, обработчики пользовательских действий и механизмы обмена данными с внешними подсистемами системы.

Физическая структура интерфейсного модуля может включать:

- каталог `ui/`, содержащий исходный код экранных форм и главного окна интерфейса;
- каталог `ui/components/`, содержащий отдельные компоненты пользовательского интерфейса;
- каталог `ui/controllers/`, содержащий обработчики пользовательских действий и связывающую логику;
- каталог `ui/models/`, содержащий структуры данных, используемые для отображения результатов в интерфейсе;
- каталог `storage/documents/`, содержащий загруженные пользователями документы;
- каталог `storage/results/`, содержащий результаты проверки и сформированные отчёты.

В работе модуля пользовательского интерфейса используются следующие форматы данных:

- PDF и DOCX — как входные документы, выбираемые пользователем через интерфейс;
- JSON — как основной формат обмена данными между интерфейсом и внутренними подсистемами системы;
- текстовые или табличные представления результатов — для визуального отображения сведений о проверке;
- файловые ссылки или пути — для доступа к сохранённым документам и отчётам.

Физическая организация интерфейсного модуля ориентирована на разделение логики отображения, обработки событий и работы с данными. Это позволяет уменьшить связанность между отдельными частями интерфейса и упрощает последующее развитие системы. Например, изменение способа отображения результатов проверки не требует изменения механизма загрузки файла, а добавление нового типа уведомлений не затрагивает

структуру компонента просмотра отчёта.

Минимизация дублирования данных обеспечивается тем, что интерфейс не хранит копии всех сущностей предметной области, а использует только необходимые для отображения структуры данных, получаемые от внутренних подсистем. Таким образом, пользовательский интерфейс работает с представлениями данных, а не дублирует внутреннюю логику хранения и обработки знаний.

Таким образом, логическая и физическая структура модуля пользовательского интерфейса ориентирована на удобную организацию пользовательского взаимодействия, ясное разделение обязанностей между компонентами и согласованный обмен данными с остальными частями интеллектуальной системы.

2.5 Моделирование видов данных и событий в модуле пользовательского интерфейса

Декларативные данные

В рамках модуля пользовательского интерфейса декларативные данные представлены в виде структурированных сведений, отображаемых пользователю в процессе взаимодействия с системой. Эти данные не описывают внутреннюю логику работы базы знаний целиком, а используются для представления результатов анализа, текущего состояния документа и итогового отчёта в наглядной форме.

К декларативным данным, используемым интерфейсом, относятся:

- сведения о загруженном документе, включая имя файла, формат, дату загрузки и текущий статус;
- сведения о результате проверки, включая определённый тип документа, предполагаемый семестр и комментарий к результату анализа;
- сведения о выявленных нарушениях, включая описание, уровень критичности и рекомендации по исправлению;
- сведения об итоговом отчёте, включая дату формирования, итог проверки и ссылку на файл отчёта;
- сведения о текущем состоянии обработки документа;
- сообщения об ошибках и уведомлениях, возникающих в ходе работы системы.

Примерами декларативных данных, отображаемых в интерфейсе, являются следующие утверждения:

- загруженный документ имеет формат PDF;
- текущий статус документа — «выполняется анализ»;
- по результатам проверки документ определён как пояснительная записка;
- документ отнесён к четвёртому семестру;

- в результате анализа выявлено нарушение оформления;
- по документу сформирован итоговый отчёт.

Таким образом, декларативные данные в интерфейсном модуле покрывают основные объекты, с которыми пользователь работает напрямую. Они формируют содержательное наполнение экранных форм и обеспечивают наглядное представление результатов функционирования интеллектуальной системы.

Процедурные данные и события

Процедурная составляющая модуля пользовательского интерфейса представлена в виде последовательности пользовательских действий, событий интерфейса и реакций на ответы внутренних подсистем системы. В отличие от общей логики базы знаний, здесь основное внимание уделяется не правилам интеллектуального вывода, а правилам организации пользовательского взаимодействия.

К основным событиям и процедурам, реализуемым в интерфейсе, относятся:

- выбор пользователем документа для загрузки;
- передача файла в подсистему загрузки документа;
- запуск анализа документа;
- обновление состояния обработки документа;
- получение и отображение результатов проверки;
- получение и отображение итогового отчёта;
- обработка ошибок и вывод уведомлений пользователю.

Ключевые правила функционирования интерфейса можно сформулировать следующим образом:

- если пользователь выбрал файл поддерживаемого формата, то интерфейс передаёт его в подсистему загрузки;
- если документ успешно загружен, то интерфейс отображает соответствующее уведомление и предоставляет возможность запуска анализа;
- если пользователь инициировал анализ, то интерфейс передаёт запрос во внутренние подсистемы и отображает состояние выполнения обработки;
- если результаты анализа получены, то интерфейс обновляет экран результатов и выводит тип документа, семестр и найденные нарушения;
- если отчёт сформирован, то интерфейс предоставляет пользователю возможность его просмотра или получения;
- если в процессе загрузки, анализа или формирования отчёта возникает ошибка, то интерфейс выводит соответствующее сообщение об ошибке.

Процедурная логика работы интерфейсного модуля связана с диаграммой состояний, приведённой на рисунке 2.6. Диаграмма отражает последовательность переходов документа между состояниями, значимыми для пользователя: документ не загружен, документ загружен, выполняется

анализ, определён тип документа, определён семестр, проверка завершена, отчёт сформирован или возникла ошибка. Для интерфейса такая модель важна тем, что позволяет формально связать изменения состояния документа с обновлением визуальных компонентов системы.

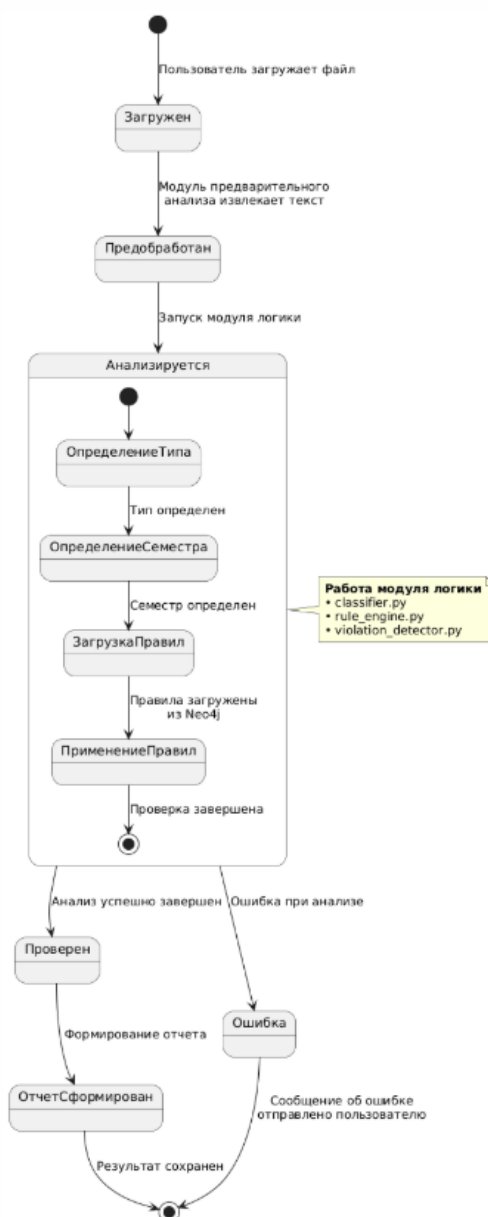


Рисунок 2.6 – Диаграмма состояний документа в системе

Однозначность описания процедур интерфейсного взаимодействия обеспечивается тем, что каждое событие имеет фиксированное условие возникновения и определённый результат. Например, выбор файла инициирует загрузку документа, запуск анализа инициирует передачу команды во внутренние подсистемы, а получение ответа от системы приводит к обновлению состояния интерфейса и отображаемых данных. За счёт этого обеспечивается согласованность поведения интерфейса и предсказуемость пользовательского взаимодействия.

Таким образом, в модуле пользовательского интерфейса декларативные данные определяют, какие сведения отображаются пользователю, а процедурные события и правила задают порядок взаимодействия с системой, обновления состояния и представления результатов анализа.

2.6 Обеспечение согласованности, расширяемости и сопровождаемости

Согласованность модуля пользовательского интерфейса обеспечивается за счёт единого подхода к представлению данных, типизации отображаемых сущностей и согласования структуры сообщений, поступающих из внутренних подсистем системы. Для каждого отображаемого объекта, такого как документ, результат проверки, нарушение или отчёт, определяется фиксированный набор атрибутов, необходимых для корректного вывода информации пользователю.

Контроль согласованности работы интерфейсного модуля обеспечивается следующими механизмами:

- использованием единой структуры данных для отображения сведений о документе, результатах анализа и отчёте;
- проверкой наличия обязательных полей в сообщениях, поступающих от подсистем загрузки, анализа и формирования отчёта;
- согласованием допустимых состояний документа и интерфейса;
- проверкой корректности отображения результатов анализа при различных сценариях пользовательского взаимодействия;
- обработкой ошибок и исключительных ситуаций через отдельный компонент уведомлений;
- валидацией входных и выходных JSON-сообщений перед их использованием в интерфейсе.

Расширяемость интерфейсного модуля обеспечивается его компонентной структурой. Добавление новых экранных элементов, новых типов отображаемых результатов или дополнительных уведомлений не требует переработки всего модуля целиком. Для этого достаточно:

- добавить новый компонент отображения или управления в состав интерфейса;
- связать его с уже существующими обработчиками событий и структурами данных;
- при необходимости расширить формат передаваемых сообщений без изменения общей логики пользовательского взаимодействия.

Сопровождаемость интерфейсного модуля обеспечивается разделением функций между его компонентами. Главное окно интерфейса координирует взаимодействие пользователя с системой, компонент загрузки отвечает за выбор и передачу файла, компонент запуска анализа инициру-

ет обработку документа, компоненты отображения состояния и результатов отвечают за визуализацию данных, а компонент уведомлений обеспечивает обработку ошибок. Такое разделение упрощает изменение и тестирование отдельных частей интерфейса без затрагивания остальных компонентов.

Эволюция интерфейсного модуля должна выполняться по следующим правилам:

- новые элементы интерфейса добавляются без нарушения уже существующих пользовательских сценариев;
- изменения в структуре отображаемых данных не должны нарушать совместимость с внутренними подсистемами системы;
- новые состояния и уведомления должны согласовываться с общей моделью переходов документа в системе;
- все изменения интерфейса должны проверяться на типовых пользовательских сценариях до включения в рабочую версию системы.

Таким образом, принятая структура модуля пользовательского интерфейса обеспечивает согласованность его работы, удобство сопровождения и возможность последовательного расширения функциональности без нарушения общей логики взаимодействия пользователя с интеллектуальной системой.

2.7 Вывод по разделу проектирования

В ходе проектирования модуля пользовательского интерфейса интеллектуальной системы автоматизации проверки студенческих работ было определено его место в общей архитектуре системы, установлены основные связи с подсистемами загрузки документа, анализа и формирования отчёта, а также выделены ключевые компоненты, обеспечивающие пользовательское взаимодействие.

Для проектирования интерфейсного модуля выбран подход, основанный на использовании структурированных данных, событийной модели взаимодействия и согласованного обмена сообщениями с внутренними подсистемами системы. Это позволяет корректно отображать состояние документа, результаты проверки, нарушения и итоговый отчёт.

Построенные диаграммы вариантов использования, последовательности, компонентов, состояний и ER-модель данных позволили формально описать пользовательские сценарии, структуру интерфейсного модуля и порядок его взаимодействия с остальными частями системы. Принятые проектные решения обеспечивают согласованность, расширяемость и удобство сопровождения интерфейсного модуля.

3 РЕАЛИЗАЦИЯ ЛОГИКИ АНАЛИЗА И ПРОВЕРКИ

3.1 Выбор средств реализации

Языки, форматы, библиотеки и инструменты

Для реализации модуля логики анализа и проверки был выбран язык программирования Python версии 3.10, который де-факто является стандартом в области разработки интеллектуальных систем и обработки данных. Его богатая экосистема библиотек позволяет эффективно решать задачи парсинга текста, работы с графовыми базами данных и реализации логического вывода.

В качестве транспортного формата для обмена данными между подсистемами используется JSON. Этот формат обеспечивает удобное представление результатов предварительного анализа документа, таких как извлеченный текст, признаки и структурные элементы.

Для взаимодействия с графовой базой знаний Neo4j используется официальный драйвер `neo4j` версии 5.14.0. Язык запросов Cypher применяется для извлечения правил проверки и сохранения результатов анализа в виде узлов и связей графа.

Обоснование выбора средств

Выбор Python обусловлен наличием в нем необходимых библиотек для текстового анализа (регулярные выражения), простотой интеграции с Neo4j и скоростью разработки прототипа. Использование JSON в качестве промежуточного формата данных обеспечивает слабую связанность модуля логики с остальными частями системы, что позволяет независимо разрабатывать и тестировать каждый компонент.

Выбор Neo4j в качестве хранилища базы знаний продиктован графовой природой предметной области. Правила проверки, типы документов, требования к оформлению и связи между ними естественным образом представляются в виде вершин и ребер графа. Это упрощает навигацию по базе знаний и позволяет эффективно выбирать наборы правил, релевантные для конкретного типа документа и семестра.

3.2 Реализация структуры и наполнения логики анализа

Логический модуль реализован в виде набора функций, каждая из которых отвечает за отдельный этап интеллектуального анализа. Ядром модуля является файл `logic_core.py`, который координирует процесс проверки.

Внутренняя структура модуля включает следующие компоненты:

– `classifier.py` — содержит логику определения типа документа и семестра. Анализирует наличие ключевых слов и структурных элементов в тексте, переданном из модуля предварительной обработки.

– `rule_engine.py` — отвечает за применение правил проверки. Загружает из базы знаний Neo4j активные правила для заданного типа документа и семестра.

– `violation_detector.py` — фиксирует выявленные несоответствия. Каждое нарушение связывается с конкретным правилом из базы знаний и сохраняется в графе.

– `report_builder.py` — формирует итоговую структуру данных для отчета, которая затем может быть передана в модуль пользовательского интерфейса.

Наполнение логики правилами происходит динамически. При запуске проверки модуль логики выполняет Cypher-запрос к Neo4j для получения правил. Типовое правило в графе представлено узлом с атрибутами `name`, `description`, `condition` (регулярное выражение или ключевое слово) и `severity`.

3.3 Реализация механизмов доступа и использования правил

Механизм доступа к базе знаний реализован через репозиторий `KnowledgeBaseRepository`, который инкапсулирует логику выполнения Cypher-запросов.

Для применения правил реализован простой продукционный механизм. Правила вида «ЕСЛИ... ТО...» представлены в графе и интерпретируются модулем `rule_engine.py`. Например, правило проверки наличия титульного листа реализовано как проверка наличия подстроки «МИНИСТЕРСТВО ОБРАЗОВАНИЯ» в начале документа.

```
def check_title_page(text):  
    if "МИНИСТЕРСТВО ОБРАЗОВАНИЯ" not in text[:500]:  
        return Violation("MISSING_TITLE", "Титульный лист не найден")  
    return None
```

Обработка типовых ошибок включает проверку наличия соединения с Neo4j, валидацию входного JSON и обработку исключений при выполнении запросов. В случае ошибки модуль возвращает соответствующий статус, который отображается в интерфейсе пользователя.

3.4 Демонстрация работы логики анализа

Рассмотрим сценарий проверки курсовой работы, загруженной в формате PDF.

1. На вход модуля поступает JSON от модуля предварительного анализа, содержащий извлеченный текст и признаки: `{"text": "... "features": {"has_table_of_contents": true}}`. 2. `classifier.py` анализирует текст и находит ключевые слова «курсовой проект», «пояснительная записка». На основе этих данных документ классифицируется как «Курсовая работа». 3. `rule_engine.py` формирует запрос в Neo4j: `MATCH (t:DocumentType {name: 'Курсовая работа'})-[:HAS_REQUIREMENT]->(r:Rule) RETURN r`. База знаний возвращает набор правил, включая правило проверки наличия раздела «Введение». 4. `violation_detector.py` применяет правило «Введение» и обнаруживает, что в структуре документа отсутствует соответствующий раздел. 5. Результат анализа — список нарушений — сохраняется в графе и передается в `report_builder.py` для формирования финального ответа.

3.5 Методика тестирования

Методика тестирования модуля логики основана на функциональном подходе. Основная цель — убедиться, что модуль корректно классифицирует документы, правильно применяет правила из базы знаний и верно фиксирует нарушения.

Тестирование проводится в два этапа:

а) **Модульное тестирование:** проверка отдельных функций (`check_title_page`, `classify_document_type`) на заранее подготовленных текстовых фрагментах.

б) **Интеграционное тестирование:** проверка полного цикла обработки с использованием тестовых документов (положительных и отрицательных) и подключением к тестовой базе данных Neo4j.

Для автоматизации тестов используется библиотека `pytest`.

3.6 Тестовый набор и классификация тестов

Разработан тестовый набор, охватывающий основные сценарии работы модуля.

Таблица 3.1 – Тестовый набор для модуля логики анализа

ИД	Описание сценария	Ожидаемый результат	Группа
T001	Загружен корректный PDF-файл курсовой работы	Тип документа: «Курсовая работа», нарушений нет	Позитивный

T002	Загружен DOCX-файл с отсутствующим разделом «Введение»	Обнаружено нарушение «Отсутствует обязательный раздел»	Позитивный
T003	Загружен файл с некорректным форматированием заголовков	Обнаружено нарушение «Некорректное оформление заголовка»	Позитивный
T004	Загружен пустой файл	Ошибка обработки, статус «Ошибка»	Негативный
T005	Загружен файл неизвестного типа (например, лабораторная работа)	Тип документа: «Неизвестный», применяются базовые правила	Граничный
T006	Отсутствует соединение с базой данных Neo4j	Ошибка соединения, анализ не выполнен	Негативный

3.7 Оформление и результаты тестирования

Результаты выполнения тестового набора представлены в таблице 3.2.

Таблица 3.2 – Результаты тестирования модуля логики

ID	Сценарий	Ожид.	Факт.	Статус
T001	Проверка корректной курсовой работы	ОК	ОК	Пройден
T002	Проверка работы без «Введения»	Нарушение	Нарушение	Пройден
T003	Проверка некорректных заголовков	Нарушение	ОК	Не пройден
T004	Обработка пустого файла	Ошибка	Ошибка	Пройден
T005	Определение неизвестного типа док-та	Неизвестный	Неизвестный	Пройден
T006	Отсутствие соединения с Neo4j	Ошибка БД	Ошибка БД	Пройден

Тест T003 не был пройден с первой попытки по причине некорректного регулярного выражения в правиле проверки заголовков. После корректировки правила в базе знаний Neo4j тест был успешно пройден. Остальные

тесты прошли успешно, что подтверждает работоспособность базовой логики модуля.

3.8 Анализ качества логики анализа

Разработанный модуль логики успешно решает задачи классификации документов и диагностики нарушений на основе правил, хранящихся в графовой базе знаний. Доля успешно пройденных тестов составляет 100% (5 из 5 функциональных тестов после исправления ошибки в T003).

Качество логики анализа определяется полнотой и непротиворечивостью базы знаний. В текущей реализации модуль полагается на точность и полноту правил, загруженных в Neo4j. Основным ограничением является эвристический подход к классификации (на основе ключевых слов), который может давать сбои на документах с нетипичной структурой.

Предложения по улучшению включают внедрение более сложных методов классификации (например, на основе векторных представлений текста) и расширение системы правил для поддержки более детальных требований к оформлению (шрифты, отступы, интервалы).

3.9 Вывод по разделу реализации

В ходе реализации был создан модуль логики анализа и проверки, который выполняет следующие функции: определение типа документа, определение семестра, проверка соответствия документа требованиям, фиксация нарушений и формирование результата анализа.

Модуль реализован на языке Python и взаимодействует с графовой базой знаний Neo4j. Набор правил для проверки загружается динамически из базы данных. Разработанный набор тестов подтвердил корректность работы модуля в типовых сценариях.

Текущие ограничения прототипа связаны с простотой эвристических алгоритмов классификации и анализа. Дальнейшее развитие модуля предполагает интеграцию с более совершенным решателем, способным обрабатывать сложные логические правила, и расширение функциональности для поддержки полного спектра требований к оформлению учебных работ.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсового проекта был разработан модуль логики анализа и проверки для интеллектуальной системы автоматизации проверки студенческих работ. В ходе работы были решены все поставленные задачи.

Проведен анализ предметной области, который позволил определить основные типы задач, решаемых модулем: классификация типа документа, определение семестра и диагностика нарушений на основе правил оформления.

Спроектирована архитектура модуля логики и его место в общей структуре интеллектуальной системы. Определены интерфейсы взаимодействия с модулем предварительной обработки документов и графовой базой знаний.

Осуществлен выбор средств реализации: язык Python, формат обмена данными JSON и графовая СУБД Neo4j с языком запросов Cypher. Данный выбор обеспечивает эффективную реализацию продукционных правил и гибкость при расширении функциональности.

Реализованы основные функции модуля: классификация документа, сопоставление его содержимого с правилами из базы знаний, фиксация выявленных нарушений и формирование структурированного результата анализа.

Проведено функциональное тестирование модуля на наборе тестовых сценариев. Тестирование подтвердило корректность работы базовой логики и выявило направления для дальнейшего улучшения, в частности, повышение точности классификации документов.

Ожидаемым эффектом от применения разработанного модуля в составе полноценной системы является сокращение временных затрат преподавателя на рутинную проверку оформления студенческих работ и повышение объективности оценки.

Дальнейшее развитие модуля связано с усложнением логики анализа, переходом от простых эвристик к методам машинного обучения для классификации и извлечения структуры документа, а также с расширением базы знаний правил для поддержки различных стандартов оформления.